

Software Requirements Specification (SRS)

Project: Project Management System (PMS) **Architecture:** Laravel API Backend + jQuery SPA Frontend

Document Version: 5.0 (Corrected & Exhaustive Definition)

Overview: This document serves as the absolute master specification for the PMS. It integrates all Phase 1, 2, and 3 requirements, details the SPA module interactions, and defines a strict, 100% Laravel-compliant database schema specifically structured for agentic IDE ingestion.

1. System Architecture & Tech Stack

- **Backend:** Laravel (Latest). Acts as a stateless API for the frontend and handles OAuth via Laravel Socialite. Uses **Laravel Sanctum** for SPA cookie-based authentication.
- **Frontend:** Single Page Application (SPA) utilizing a single `index.html` container. Content and states are dynamically loaded via jQuery and AJAX.
- **Database:** MySQL, strictly adhering to Laravel Eloquent ORM conventions. Soft deletes implemented for critical records.
- **Authentication:** OAuth 2.0 (Google & Microsoft) exclusively. No local passwords.
- **Email Services:** PHP Inbuilt Mail Function / Laravel Mailables.

2. Exhaustive Module Breakdown

2.1 Authentication & Profile Module (Phase 1)

- **Strict OAuth Entry:** No local registration forms. Users authenticate strictly via Google/Microsoft redirects.
- **Auto-Provisioning:** On callback, the system checks if `provider_id` or `email` exists. If not, it creates a new user, syncing Name, Email, and Avatar URL. Users are assigned the default role of "Team Member".
- **SPA Session:** Login/Logout is handled via AJAX (Sanctum SPA Auth) to maintain the single-page experience.
- **Roles:** Admin (creates projects), Project Manager (task assignments), Team Member (execution/self-assignment).

2.2 Project Management Module (Phase 1)

- **Core Container:** Admins define Projects (Name, Description, Start/End Dates, Status).

- **Team Mapping:** Admins assign users to specific projects (managed via the `project_user` pivot table).

2.3 Jira-Style Backlog Module (Phase 1)

- **The Backlog:** A distinct view showing all tasks where `is_backlog = true`.
- **Prioritization:** Users can drag-and-drop tasks within the backlog to set priority (updates `position` via AJAX).
- **Self-Assignment ("Claiming"):** Team Members click "Claim" on a backlog item. This triggers a backend update setting `assigned_to = {user_id}`, `is_backlog = false`, and `status = 'To Do'`. The task instantly moves to the Kanban board.

2.4 Visual Kanban Board Module (Phase 1)

- **Columns:** Three distinct states: "To Do", "In Progress", and "Done".
- **No-Reload Updates:** Moving a card between columns triggers a `PUT /tasks/{id}/status` request. The UI updates instantly on success.

2.5 Communication & Tracking Modules (Phase 2 & 3)

- **Time Tracking (Phase 2):** Tasks hold an `original_estimate`. Users log actual hours worked, updating `total_spent`.
- **Collaboration (Phase 2):** Threaded comments attached to tasks. Task attachments for sharing files.
- **Activity Logging (Phase 2):** Automated system logs tracking when a task is claimed or changes status (records old and new values).
- **Notifications (Phase 3):** Internal app alerts and external email triggers for task assignments or mentions.

3. Comprehensive Database Schema (Laravel Eloquent Standard)

All tables automatically include `$table->timestamps()` (`created_at` and `updated_at`).

3.1 Identity & Access

Column Name	Laravel Migration Definition	Description
Table: users		(Note: No password fields as auth is strictly OAuth)
<code>id</code>	<code>\$table->id();</code>	Primary Key
<code>name</code>	<code>\$table->string('name');</code>	From OAuth

Column Name	Laravel Migration Definition	Description
email	<code>\$table->string('email')->unique();</code>	From OAuth
email_verified_at	<code>\$table->timestamp('email_verified_at')->nullable();</code>	Standard Laravel
remember_token	<code>\$table->rememberToken();</code>	Standard Laravel
provider_name	<code>\$table->string('provider_name')->nullable();</code>	'google' or 'microsoft'
provider_id	<code>\$table->string('provider_id')->nullable();</code>	OAuth unique ID
avatar_url	<code>\$table->string('avatar_url')->nullable();</code>	Profile image URL
role_id	<code>\$table->foreignId('role_id')->default(3)->constrained();</code>	FK to roles. Default 3 (Team Member)
status	<code>\$table->enum('status', ['active', 'inactive'])->default('active');</code>	Account status

Column Name	Laravel Migration Definition	Description
Table: roles		
id	<code>\$table->id();</code>	Primary Key
name	<code>\$table->string('name');</code>	Admin, PM, Team Member

(Note: `password_reset_tokens` table was removed as it is redundant in a strict OAuth system).

3.2 Workspace Container

Column Name	Laravel Migration Definition	Description
Table: projects		
id	<code>\$table->id();</code>	Primary Key
name	<code>\$table->string('name');</code>	Project Name
description	<code>\$table->text('description')->nullable();</code>	Details
start_date	<code>\$table->date('start_date')->nullable();</code>	Timeline Start
end_date	<code>\$table->date('end_date')->nullable();</code>	Timeline End
status	<code>\$table->string('status')->default('Active');</code>	Active/Archived

Column Name	Laravel Migration Definition	Description
<code>created_by</code>	<code>\$table->foreignId('created_by')->constrained('users');</code>	Creator ID
<code>deleted_at</code>	<code>\$table->softDeletes();</code>	Soft delete support

Column Name	Laravel Migration Definition	Description
Table: project_user (Pivot)		
<code>project_id</code>	<code>\$table->foreignId('project_id')->constrained()->cascadeOnDelete();</code>	FK to projects
<code>user_id</code>	<code>\$table->foreignId('user_id')->constrained()->cascadeOnDelete();</code>	FK to users

3.3 Agile Workflows & Tracking

Column Name	Laravel Migration Definition	Description
Table: tasks		
<code>id</code>	<code>\$table->id();</code>	Primary Key
<code>project_id</code>	<code>\$table->foreignId('project_id')->constrained()->cascadeOnDelete();</code>	Parent Project
<code>creator_id</code>	<code>\$table->foreignId('creator_id')->constrained('users');</code>	Task Creator
<code>assigned_to</code>	<code>\$table->foreignId('assigned_to')->nullable()->constrained('users');</code>	Null if in backlog
<code>title</code>	<code>\$table->string('title');</code>	Summary
<code>description</code>	<code>\$table->text('description')->nullable();</code>	Details
<code>status</code>	<code>\$table->string('status')->default('To Do');</code>	To Do, In Progress, Done
<code>priority</code>	<code>\$table->string('priority')->default('Medium');</code>	High, Medium, Low
<code>due_date</code>	<code>\$table->date('due_date')->nullable();</code>	Deadline for the task
<code>is_backlog</code>	<code>\$table->boolean('is_backlog')->default(true);</code>	Controls board visibility

Column Name	Laravel Migration Definition	Description
<code>position</code>	<code>\$table->integer('position')->default(0);</code>	Drag-and-drop order
<code>original_estimate</code>	<code>\$table->decimal('original_estimate', 8, 2)->nullable();</code>	Hours (Phase 2)
<code>total_spent</code>	<code>\$table->decimal('total_spent', 8, 2)->default(0);</code>	Hours (Phase 2)
<code>deleted_at</code>	<code>\$table->softDeletes();</code>	Soft delete support

Column Name	Laravel Migration Definition	Description
Table: task_comments (Phase 2)		
<code>id</code>	<code>\$table->id();</code>	Primary Key
<code>task_id</code>	<code>\$table->foreignId('task_id')->constrained()->cascadeOnDelete();</code>	FK to tasks
<code>user_id</code>	<code>\$table->foreignId('user_id')->constrained()->cascadeOnDelete();</code>	Author
<code>comment</code>	<code>\$table->text('comment');</code>	Message body

Column Name	Laravel Migration Definition	Description
Table: task_activity_logs (Phase 2)		
<code>id</code>	<code>\$table->id();</code>	Primary Key
<code>task_id</code>	<code>\$table->foreignId('task_id')->constrained()->cascadeOnDelete();</code>	FK to tasks
<code>user_id</code>	<code>\$table->foreignId('user_id')->constrained()->cascadeOnDelete();</code>	User who triggered action
<code>action</code>	<code>\$table->string('action');</code>	e.g., 'Status Changed'
<code>old_value</code>	<code>\$table->string('old_value')->nullable();</code>	e.g., 'To Do'
<code>new_value</code>	<code>\$table->string('new_value')->nullable();</code>	e.g., 'In Progress'

Column Name	Laravel Migration Definition	Description
Table: task_attachments (Phase 2)		
id	<code>\$table->id();</code>	Primary Key
task_id	<code>\$table->foreignId('task_id')->constrained()->cascadeOnDelete();</code>	FK to tasks
user_id	<code>\$table->foreignId('user_id')->constrained()->cascadeOnDelete();</code>	Uploader
file_path	<code>\$table->string('file_path');</code>	Path to stored file
file_name	<code>\$table->string('file_name');</code>	Original file name

3.4 Notifications

Column Name	Laravel Migration Definition	Description
Table: notifications (Phase 3)		
id	<code>\$table->id();</code>	Primary Key
user_id	<code>\$table->foreignId('user_id')->constrained()->cascadeOnDelete();</code>	Recipient of alert
message	<code>\$table->text('message');</code>	Alert text
is_read	<code>\$table->boolean('is_read')->default(false);</code>	Read state boolean

4. Execution Roadmap Summary

- **Phase 1:** Establish foundational database. Implement Laravel backend with Socialite for OAuth and Sanctum for SPA auth. Build the jQuery SPA interface. Connect the SPA to the API to handle Backlog population, Task Claiming, and Kanban drag-and-drop status updates.
- **Phase 2:** Expand the UI and API to support threaded comments on task modals, file attachments, implementation of actual time tracking inputs, and automated background logging of user actions on the board with old/new state comparisons.

- **Phase 3:** Finalize system loops by building the Notification panel in the SPA, integrating PHP Mail for assignment alerts, and generating administrative reports.